If for some reason something goes wrong in the worker and it throws an exception, the manager receives a timeout. In such a scenario, no response is sent from the worker due to the exception; therefore, nothing ends up in dead letters.

3.2.2 Coding the manager

The manager does the following:

1. It creates a worker for each received text and asks it to `Parse`.
2. The `ask` specifies what to do with the response:
    - On a `Success`, the manager sends a `Report` message to itself containing the worker's name.
    - On a `Failure`, the manager wraps the exception message in a report, which it again sends to itself.
3. When the manager receives a report, whether success or failure, the content is printed.

3.2.3 The protocols

The manager has the same protocol as in the previous example, but instead of the adapter, it now has `Report`, which works in combination with `ask`. `Report` is necessary to deal with the worker's response. In the callback of `ask`, the manager has to send a message to itself, and this is expressed in the `ask` signature. Before we deal with that, let's look at the manager and worker protocols to see both sides of the communication.